Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Ders 1 ile aynı gün yapılan uygulama oturumudur.
  - Asıl anlatım yaklaşık 00:30'da başlar. Öncesi NetBeans ve JDK'nın birlikte indirilip kurulmasıdır.
- Geliştirme ortamının kurulumu
  - Apache NetBeans indirildi. Kurulumda "Customize" ile gereksiz bileşenler (ör. Enterprise Edition, PHP) çıkarılabilir.
  - NetBeans için JDK gerekir. JDK yoksa kurulum Oracle'ın sayfasına yönlendirir. Derste JDK 15 indirildi.
  - Oracle'ın ücretsiz eğitim sayfaları (Oracle Academy / Java Explorer) tanıtıldı. Üyelik için `.edu` uzantılı e-posta önerildi.
- İlk Java projesi
  - File, New Project, Java with Ant, Java Application. "Create Main Class" seçilir.
  - Main sınıf: `main` metodunu içeren, programın başladığı sınıf. Buna sürücü (driver) veya test sınıfı da denir.
  - `System.out.println` ve `System.out.print` farkı: `println` satır sonuna geçer, `print` geçmez.
  - NetBeans kısayolu: `sout` yazıp Tab'a basınca `System.out.println("");` oluşur.
  - Proje klasörü: Belgeler altında `NetBeansProjects`. Kaynak kod `src` altındaki paket klasöründedir.
- Paketler (package)
  - Projeyi mantıksal olarak parçalara böler. Farklı paketlerde aynı adlı sınıflar olabilir.
  - Başka paketteki sınıfı kullanmak için o paket dahil edilir (`import`).
- İlk sınıflar: `Kitap` ve `Yazar`
  - Yazar bir `String` olarak değil, ayrı bir sınıf olarak tasarlandı.
  - `new` ile nesne oluşturma, nesneyi yazdırma, referans mantığı, çöp toplayıcı (garbage collector).
- UML eklentisi
  - Easy UML eklentisi NetBeans'e elle (`.nbm` dosyalarıyla) kuruldu. Kodu verilen sınıflardan sınıf diyagramı üretildi.
  - Bir öğrenci ekranını paylaşıp IntelliJ'deki Swing UI Designer ve PlantUML eklentisini gösterdi.
- `JFrame` ile hazır grafik arayüz formu kısaca gösterildi. Bu konu dönem sonunda işlenecek.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınıf adı ile dosya adı aynı olmalıdır. Sınıf adını değiştirmek için sağ tık, Refactor, Rename kullanın; ad koddaki her yerde değişir.
- Sınıf adlarında tekil isim kullanın: `Kitap`, `Yazar`.
- Yazarı kitabın içinde `String` olarak tutmak doğru tasarım değildir. Yazarın da özellikleri olduğu için ayrı bir sınıf olmalıdır.
- Nesne yönelimli programlama bir referans mekanizmasıdır. Her şey birbirine referans verir. Derin kopya (deep copy) maliyetlidir, gerekirse yapılır.
- Bu derste `if`, `for`, `while` tekrar anlatılmayacak. Bunlar C ile neredeyse aynıdır.
- UML çizebileceğiniz bir aracınız mutlaka olsun (NetBeans, Eclipse veya IntelliJ eklentisi).
- Hoca grafik arayüzü sürükle-bırak araçlarıyla öğrenmeye başlamayı doğru bulmuyor.

Kısa Tekrar Notları

- Yeni proje: File, New Project, Java with Ant, Java Application, Create Main Class.
- Programın çalışması `main` metodu olan sınıftan başlar. Bir uygulamada genellikle bir tane olur.
- `System.out.println`: `System` sınıfının `static` alanı olan `out` nesnesinin (`PrintStream` tipinde) `println` metodu.
- `println` metodunun `int`, `long`, `String`, `Object` gibi farklı parametreli birçok sürümü vardır. Buna metot aşırı yükleme (method overloading) denir.
- Paket: projeyi mantıksal olarak bölmek, aynı adlı sınıfları ayırt etmek için kullanılır.
- `int x = 5;` değer tiplidir. `println(x)` değeri (5) yazar.
- `Yazar y = new Yazar();` referans tiplidir. Nesne yığın belleğinde (heap) durur, referansı `y` değişkeninde tutulur.
- Nesneyi doğrudan yazdırınca `paket.SinifAdi@...` biçiminde bir metin çıkar (derste `javaapplication3.Yazar@...`).
- `k2 = k;` yeni nesne oluşturmaz. Yalnız referans atanır, iki değişken aynı nesneyi gösterir. Bu kopyalama değildir.
- Hiçbir referansın göstermediği nesneyi Java'nın çöp toplayıcısı temizler. C'deki `free` gerekmez.
- Nokta operatörüyle iç içe erişim: `k.yazar.ad`.
- Easy UML: Sınıflardan diyagram üretir, diyagramdan kod üretir (code generation), koddan diyagram çıkarır (reverse engineering).

Detaylı Açıklamalar

Derste yazılan kod

- `Kitap` sınıfı anlatımdan yeniden yazıldı: ad, sayfa sayısı ve `Yazar` tipinde bir alan.

```
package javaapplication3;

public class Deneme {
    public static void main(String[] args) {
        System.out.println("deneme");
        System.out.print("denemeee");   // alt satıra geçmez
        System.out.println("sdfsdf");
        int x = 5;
        System.out.println(x);          // 5
        Yazar y = new Yazar();
        System.out.println(y);          // javaapplication3.Yazar@... gibi bir çıktı
        Kitap k = new Kitap();
        System.out.println(k);
        // k.yazar.ad ile kitabın yazarının adına erişilir
    }
}

public class Yazar {
    String ad;
    int id;
}

public class Kitap {
    String ad;
    int sayfaSayisi;
    Yazar yazar;   // yazar ayrı bir sınıf
}
```

- Hoca ileride bir kitabın birden fazla yazarı olabileceği için yazarı bir dizi veya liste olarak tutmayı da gösterdi. Sonra konuyu karıştırmamak için tek yazarla devam etti.
- Erişim belirteçleri (`private`, `public`) henüz yazılmadı. Sonraki derslerde eklenecek.

Bellekte nesneler: stack ve heap

- `new` ile oluşturulan nesne belleğin yığın (heap) bölgesinde durur.
- Değişken (`y`, `k`) yerel değişken olarak stack bölgesindedir ve nesnenin referansını tutar. Bir anlamda işaretçi gibidir.
- Kitap nesnesinin içinde yazar alanı vardır. Bu alan da ayrı bir yazar nesnesini gösterir. Somut bir kitap için önce somut bir yazar nesnesi gerekir.
- Örnek senaryo:
  - `Kitap k = new Kitap();` heap'te bir kitap nesnesi oluşturur.
  - `Kitap k2 = k;` yeni nesne oluşturmaz. `k2` aynı nesneyi gösterir.
  - `Kitap k3 = new Kitap();` ikinci bir nesne oluşturur.
  - `k3 = k;` yapılınca ikinci nesneyi gösteren referans kalmaz. Çöp toplayıcı bu alanı bir süre sonra serbest bırakır.

Paketler

- Örnek: Bir üniversite yazılımında muhasebe ve öğrenci işleri ayrı paketlerde tutulur.
- İki pakette de `Yazdir` adlı bir sınıf olabilir. Paket adı sayesinde birbirine karışmazlar.
- Tam ad: `javaapplication3.Deneme` gibi, paket adı ve sınıf adı.

Debug

- Satır numarasının yanına tıklanarak kesme noktası (breakpoint) konur.
- Debug menüsünden Debug Project ile program adım adım çalıştırılır.

Ders sonrası notlar

- **`System.out` bir iç sınıf değildir:** Hoca önce `out` için "inner class" dedi, sonra kaynak koda bakıp kendisi düzeltti. `out`, `System` sınıfının `public static final PrintStream` tipindeki alanıdır.
- **Nesneyi yazdırınca çıkan metin:** `paket.SinifAdi@16db9742` gibi çıktıdaki sayı nesnenin bellek adresi değildir. `Object.toString()` metodu sınıf adını ve nesnenin `hashCode()` değerini onaltılık olarak yazar.
- **`clone` derin kopya yapmaz:** Hoca derin kopya için `clone` metodu olduğunu, bu metodun "bütün değerleri bit bit kopyaladığını" söyledi. `Object.clone()` varsayılan olarak **sığ kopya** yapar. Nesnenin içindeki referans alanları kopyalanmaz; kopya ile asıl nesne aynı iç nesneleri gösterir. Derin kopya için `clone` metodu override edilip iç nesneler de ayrıca kopyalanmalıdır. Java ile denendi: `kopya == b` sonucu `false`, `kopya.yazar == b.yazar` sonucu `true` çıktı.
- **`String` temel tip değildir:** Hoca nesnelerin "`string`, `integer`, `char` gibi" temel veri tiplerinden oluştuğunu söyledi. `String` bir sınıftır. Java ile denendi: `String.class.isPrimitive()` sonucu `false` çıktı.
- **Dosya adı kuralı:** Dosya adı ile aynı olma zorunluluğu `public` sınıflar içindir. Bir dosyada `public` olmayan başka sınıflar da olabilir.
- **Döngü değişkeni:** Hoca C'de `for (int i = 0; ...)` yazmayın demişti. C99 ve sonrası bu yazımı destekler. Eski derleyici ayarlarında hata verebilir. Java'da her zaman geçerlidir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.